RéaMix 2025-26 : Réalité Mixte pour la Formation et l'Ingénierie

BRIEAU Eric-Nicolas, HAMDY Maya, LEFEBVRE Eric,
MOQUEREAU Clement, OLIVIER Matéo, SARKISSIAN Enzo

Année universitaire : 2025-2026

Tuteurs université :
MAY Madeth,
COUPRY Corentin
madeth.may@univ-lemans.fr,
corentin.coupry@univ-lemans.fr

Résumé

Ce rapport présente le développement de notre projet de quatrième année réalisé dans le cadre du projet RéaMix 2025-26, consistant en la création d'un jeu en réalité virtuelle destiné à l'initiation aux mécaniques fondamentales de la VR. Le joueur incarne un enfant évoluant dans une garderie transformée en centre de test, guidé par une intelligence artificielle à travers plusieurs salles et mini-jeux. L'objectif est de familiariser progressivement l'utilisateur aux interactions en réalité virtuelle telles que le déplacement, la saisie d'objets, les interactions physiques et la résolution de simples d'énigmes.

Le projet repose sur une collaboration entre la modélisation 3D sous *Blender* et le développement sous *Unity* 3D via le *XR Interaction Toolkit*. Une attention particulière a été portée à l'identité visuelle du jeu, mélangeant une esthétique mi-cartoon mi-réaliste, ainsi qu'aux problématiques d'immersion et de confort utilisateur liées à la réalité virtuelle, notamment la gestion du *motion sickness*, des collisions et des retours visuels, sonores et haptiques.

Le document détaille l'ensemble du cycle de développement : organisation du projet, création des assets, architecture logicielle basée sur un système d'événements, conception des interfaces VR et implémentation des différents mini-jeux. Parmi ceux-ci figurent un jeu de lancer de balles dans un panier, un *Whack-a-Mole*, une slot machine ainsi qu'un *Simon Says* situé dans une seconde salle inspirée des univers des *Backrooms* et des aires de jeux pour enfants.

Le résultat final est une démonstration jouable mettant en avant des mécaniques VR variées, conçue pour être accessible à des utilisateurs novices tout en proposant une expérience immersive.

Table des matières

1 Contexte **4**
1.1 Présentation de Réamix . . . 4
1.2 Choix du thème . . . 4
1.3 Inspirations . . . 4
1.4 Scénario . . . 5
1.5 Choix de l'aspect esthétique du jeu . . . 6

2 Organisation **8**
2.1 Phase pré-JPO . . . 8
2.2 Phase post-JPO . . . 9
2.3 Outils de gestion de projet . . . 10
2.4 Répartition des rôles . . . 10

3 Cycles de développement **11**
3.1 Développement sur Blender . . . 11
3.2 Développement Unity . . . 14
3.2.1 Game Design et progression du joueur . . . 14
3.2.2 Mini-jeux et implémentation technique . . . 14
3.2.3 Level Design : conception et refonte de la deuxième salle . . . 16
3.2.4 Architecture du projet et outils VR . . . 17
3.2.5 Pipeline de création et intégration des assets . . . 17
3.2.6 Système d'événements et architecture logicielle . . . 19
3.2.7 Menu et interfaces VR . . . 20
3.2.8 Sound Design . . . 22

4 Contraintes et immersion **22**
4.1 Déplacements et confort utilisateur . . . 23
4.2 Retours visuels, sonores et haptiques . . . 23
4.3 Interactions physiques et immersion . . . 24

5 Retours utilisateurs **24**
5.1 Déplacements et confort . . . 24
5.2 Guidage et prise en main . . . 25
5.3 Mini-jeu du panier . . . 25
5.4 Interactions et objets . . . 25
5.5 Ambiance et level design . . . 25

6 Difficultés rencontrées **26**
6.1 Exports . . . 26
6.2 Connexion avec le casque . . . 26
6.3 Erreurs de compilation . . . 27
6.4 Conflits . . . 27

7 Étude d'impact environnemental **27**

8 Conclusion **28**

1 Contexte

1.1 Présentation de Réamix

Le projet RéaMix s'inscrit dans la démarche compétence initiée par l'ENSIM. Son objectif principal est de promouvoir la recherche et développement (R&D) d'applications en Réalité Mixte dédiées à la formation, tout en préparant les futurs ingénieurs de l'ENSIM à la maîtrise des technologies émergentes.

Ce projet a quatre thématiques possibles :

— jeu vidéo ;

— formation ;

— commerce numérique ;

— médecine.

L'équipe conçoit et développe une application VR pouvant être valorisée dans divers contextes : démonstrations, forums, portes ouvertes, challenges numériques, etc.

Dans une seconde phase, le projet met l'accent sur la créativité et la mise en œuvre des éléments définis lors de la phase de conception.

Les domaines techniques exploités dans le cadre du projet comprennent :

— modélisation 3D (Blender) ;

— *Unity* 3D ;

— réalité virtuelle et réalité augmentée.

1.2 Choix du thème

Le choix de ce projet s'est porté sur la création d'un jeu vidéo d'initiation et de tutoriel à la réalité virtuelle. L'objectif principal est de permettre à un utilisateur novice de découvrir et de maîtriser les mécaniques de base de la VR (se déplacer, saisir des objets, interagir avec l'environnement). Puisque le joueur doit réaliser des actions basiques pour se familiariser avec son environnement et en quelque sorte son nouveau "corps virtuel", nous avons naturellement choisi un thème enfantin. En faisant incarner un enfant à notre joueur, nous intégrons l'apprentissage de la VR à notre scénario. Les erreurs du joueur liées à cet apprentissage pourront donc être prises en compte par l'environnement pour améliorer l'immersion.

1.3 Inspirations

Avant de nous lancer directement dans le développement, nous avons d'abord cadré nos idées et nous nous sommes assurés d'être tous sur la même longueur d'onde en faisant un *moodboard*. Cela consiste en un rassemblement de références à des jeux auxquels nous avons joué, d'ambiances sonores, de visuels et autres idées que nous souhaitions implémenter dans le jeu. Voici quelques points qui en sont ressortis :

— Faire un jeu à l'image de *Portal*, avec des salles de test progressivement plus complexes (image de gauche sur la Figure 1).

— Le narrateur omniscient de *The Stanley Parable* et son illusion du choix dans la progression (Figure 2).
— Des jeux pour enfant comme le *Simon Says* ou le *Whack-A-Mole* (jeu où il faut taper des taupes avec un marteau).

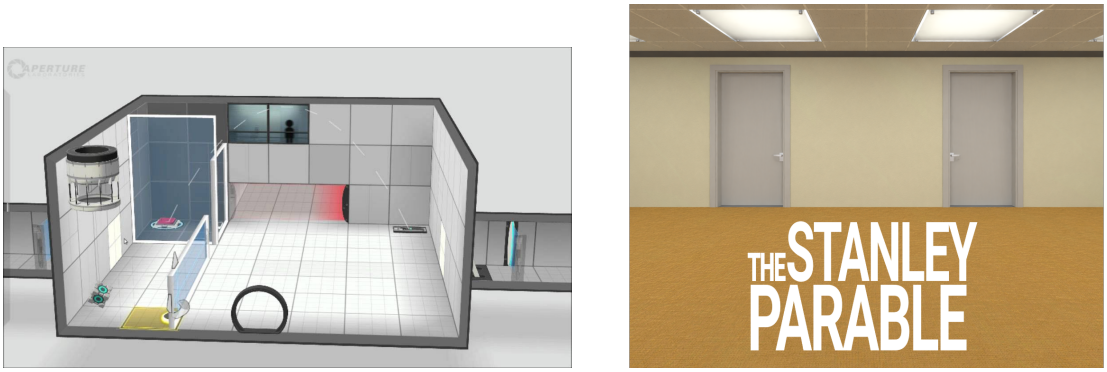


FIGURE 1 – Exemple d'inspirations présentes sur notre moodboard

Pour la deuxième salle, nous nous sommes inspirés de deux univers visuels distincts. D'un côté, les *Backrooms* : un concept issu de la culture Internet qui décrit des espaces intérieurs infinis, vides et légèrement inquiétants, avec des couloirs jaunes et un éclairage blafard. De l'autre, les aires de jeux pour enfants, avec leurs couleurs vives et leurs structures géométriques. Ce mélange entre l'ambiance légèrement oppressante des *Backrooms* et l'univers coloré des jeux d'enfants s'inscrit naturellement dans le ton de notre jeu, à la fois enfantin et mystérieux.



FIGURE 2 – Inspirations pour la salle 2

1.4 Scénario

Plutôt que de proposer un simple bac à sable technique, nous avons choisi de plonger notre joueur dans une histoire qu'il devra comprendre et essayer de résoudre par lui-même. Le jeu prend place dans une série de chambres pour enfant qui sont en réalité des salles de test. Chaque salle vise à tester, grâce à un petit jeu, les capacités cognitives, physiques et motrices de l'enfant qui les traversera. Le joueur incarne un jeune enfant guidé à travers ces salles par une intelligence artificielle via des commandes vocales. Le but de cette

IA est d'entraîner ces enfants pour les rendre obéissants. Ce n'est en réalité pas le seul enfant qu'elle entraîne, elle en possède des milliers et si le joueur ne prend pas les bonnes décisions, l'IA pourrait bien tenter de se débarrasser de cet enfant désobéissant. Le joueur est donc libre d'écouter ou non les ordres de l'IA.

Dans la version finale du jeu, le déroulement est le suivant : le personnage principal commence dans un parc pour bébé. Le joueur doit réussir à ouvrir une porte pour en sortir. Ensuite, il doit réussir un jeu qui consiste à lancer un certain nombre de balles dans un panier placé un peu plus loin. Ces balles sortent une par une d'un tube placé dans le mur. Le jeu suivant est une *Slot Machine* : le joueur actionne une manivelle, ce qui fait apparaître un mot aléatoire sur la machine. Il doit ensuite retrouver les cubes de l'alphabet éparpillés dans la salle pour épeler ce mot, ce qui déverrouille une porte menant à la partie suivante de la salle, où l'attend un *Whack-a-Mole*. Le marteau se trouve à côté du jeu sur une petite table d'enfant. Lorsque le joueur gagne, une transition mène vers la deuxième salle, aux inspirations et à l'atmosphère bien différentes de la première.

Une fois la transition effectuée, le joueur se retrouve dans la deuxième salle. Il doit d'abord traverser un labyrinthe à l'ambiance inspirée des *Backrooms* : couloirs répétitifs, éclairage blafard et atmosphère légèrement oppressante. À la sortie du labyrinthe, une cabine d'ascenseur attend le joueur et le transporte vers une aire de jeux pour enfants, lumineuse et colorée, tranchant volontairement avec l'ambiance du labyrinthe. C'est là que se déroule le dernier jeu de la démo : un *Simon Says*, où le joueur doit reproduire une séquence de couleurs / sons dans le bon ordre.

1.5 Choix de l'aspect esthétique du jeu

Pour l'identité visuelle de notre jeu, nous avons opté pour un style mi-cartoon, mi-réaliste. Le jeu prend place dans une chambre d'enfant / garderie dans laquelle des objets et un environnement interactif entourent le joueur. Nous nous sommes beaucoup inspirés des images ci-dessous pour l'ambiance générale de la salle. Par exemple, la chaise et la bibliothèque visibles sur la figure 3 ont servi de modèles pour nos modélisations de ces objets en 3D. Les murs colorés avec des dessins, les cubes, les étagères ont aussi été ajoutés dans notre jeu pour essayer de recréer une ambiance similaire à ces images.

À partir de ces bases, nous avons par la suite décidé d'enrichir l'ambiance de la première salle en y ajoutant davantage de détails et d'éléments décoratifs. L'objectif était de rendre l'environnement plus vivant et plus crédible, notamment grâce à un travail plus poussé sur l'éclairage, les objets d'arrière-plan et les petites animations d'ambiance. Ce souci du détail contribue à renforcer l'immersion dès les premières secondes du jeu.

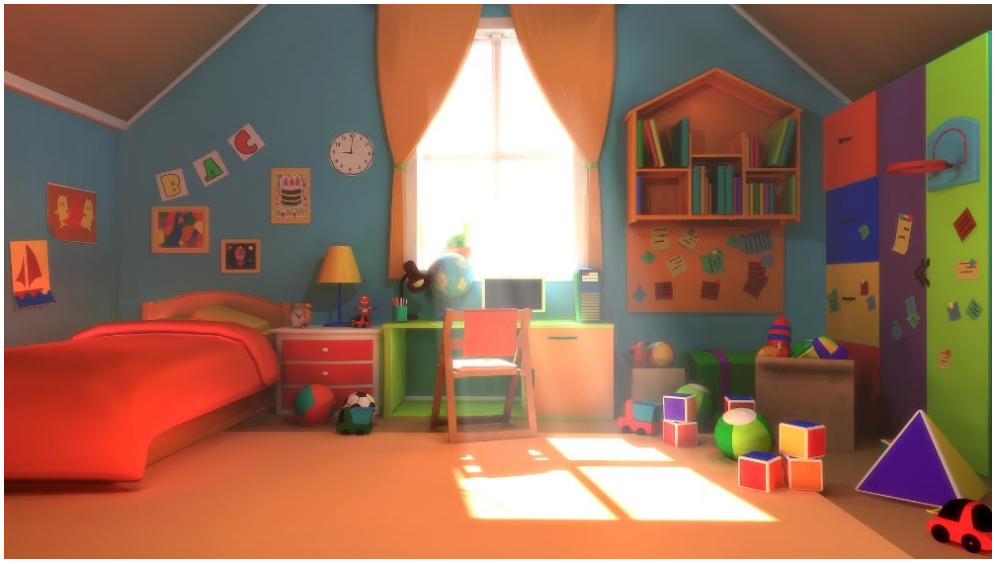


FIGURE 3 – Exemple d'inspiration 1

Le côté "cartoon" permet d'accentuer l'aspect enfantin et coloré propre à une chambre d'enfant ou une garderie, tout en rendant les objets importants (comme les balles du mini-jeu) facilement identifiables. Le côté "réaliste", quant à lui, est apporté par le travail sur les proportions et les éclairages. Ce mélange crée une ambiance immersive qui ne surcharge pas visuellement l'utilisateur, tout en conservant une certaine crédibilité physique pour ne pas briser l'immersion en réalité virtuelle.



FIGURE 4 – Exemple d'inspiration 2

2 Organisation

Lors de nos premières séances, nous avons pris le temps de définir le thème du jeu et sa direction artistique. Nous avons aussi clarifié les objectifs qu'il fallait atteindre pour considérer le projet comme terminé. En effet, ce n'est pas un projet avec un début et une fin clairs : on peut continuellement améliorer un jeu vidéo, de la même manière qu'on peut améliorer un logiciel. Nous avons donc dû fixer une limite, avec les fonctionnalités clés à inclure selon le temps estimé pour chacune. Nous avons ainsi séparé nos objectifs en deux phases : le produit minimum viable (MVP) à montrer lors des journées portes ouvertes, puis le rendu final regroupant toutes les fonctionnalités prévues.

2.1 Phase pré-JPO

La première phase de conception s'étendait du 2 octobre 2025 au 7 février 2026, sur 15 séances de 3 heures, pour un total de 45 heures. Il est important de noter qu'en dehors de ces 45 heures en présentiel à l'ENSIM, nous avons aussi beaucoup travaillé chez nous pour garantir un MVP prêt à temps pour les portes ouvertes. Cela a continué lors de la seconde phase, afin de bien intégrer toutes les parties au rendu final. Tout au long du projet, nous avons également pris sur notre temps personnel pour la rédaction du rapport, la création des diapositives de présentation, les démonstrations, ainsi que le temps passé au salon Laval Virtual.

Cette phase visait à produire un MVP représentatif des capacités de l'équipe : un casque fonctionnel avec notre application, un personnage déplaçable, un environnement cohérent et un but de jeu clair. L'enjeu était d'obtenir un jeu jouable, avec suffisamment d'éléments interactifs pour recueillir des retours concrets du public. Les visiteurs des portes ouvertes correspondaient justement à notre public cible : des personnes néophytes en réalité virtuelle, idéales pour tester l'accessibilité du jeu.

Pour nous organiser, nous avons d'abord posé les bases : s'assurer que tout le monde était d'accord sur un même thème, l'avait compris et pouvait se l'approprier pour y ajouter une touche personnelle pertinente. Nous avons ensuite découpé l'objectif de MVP en tâches atomiques que chacun pouvait réaliser indépendamment, regroupées en quatre grandes parties :

- Modélisation : la création d'objets 3D sur *Blender* ;
- Scripting : la mise en place de la VR ainsi que l'écriture de scripts en C# ;
- Histoire : l'approfondissement de l'idée initiale pour donner une identité au jeu ;
- Game design : le design d'une salle pertinente en accord avec l'histoire.

Chaque membre s'est positionné sur une ou deux de ces parties selon ses préférences et les compétences qu'il souhaitait développer. La conception d'un jeu vidéo demandant de solides connaissances dans chacun de ces domaines, nous avons choisi de nous spécialiser : cela nous a permis de maîtriser plus rapidement notre aspect respectif plutôt que de rester sur des connaissances de surface.

Concrètement, nous avancions par sprints, des cycles de 2 semaines permettant de développer des fonctionnalités de manière incrémentale. À ce stade, nous progressions tous ensemble afin de garder le contrôle sur l'ensemble des tâches en parallèle, ce qui nous permettait aussi de nous entraider rapidement dès que l'un de nous bloquait. Par exemple, pour la première salle, un sprint avait pour but de poser les fondations : textures cohérentes, premier objet modélisé et joueur sous forme de capsule déplaçable grâce au casque en VR. Toutes les 2 semaines, peu importe l'avancée de chacun, nous faisions une réunion pour faire le point sur les difficultés rencontrées et redéfinir un objectif global pour la suite.

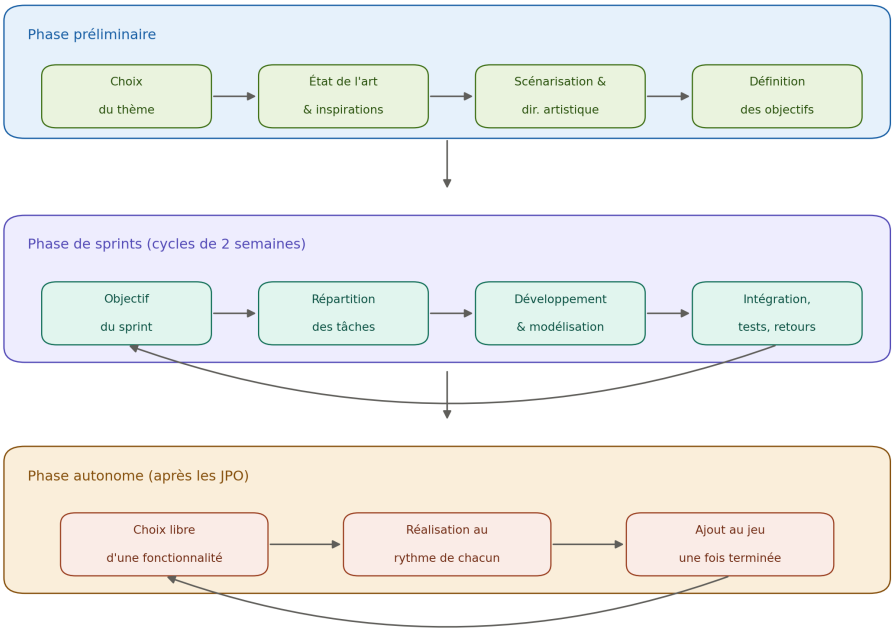


FIGURE 5 – Workflow des différentes phases du projet

2.2 Phase post-JPO

La seconde phase de conception s'étendait du 7 février au 4 juin, sur 14 séances de 3 heures, pour un total de 42 heures. Ayant rempli les objectifs de la première phase, nous avons pu recueillir de nombreux retours lors des portes ouvertes. Les profils des joueurs étant très variés, nous avons obtenu un large panel de retours, nous permettant de mieux comprendre les attentes des joueurs en termes d'accessibilité.

Nous avons globalement gardé les mêmes objectifs sur la conception du jeu, mais ces retours nous ont amenés à repositionner certaines priorités pour que le jeu corresponde mieux aux attentes des joueurs sans s'écarter de notre vision initiale. Nous avons notamment renforcé le côté initiatique du jeu en simplifiant certaines interactions et en ajoutant le plus d'indicateurs non invasifs possibles pour guider le joueur.

L'organisation du travail a elle aussi évolué. Une fois tous plus à l'aise avec les technologies et le développement de jeu vidéo en général, nous avons changé de méthode. Plutôt que d'avancer en groupe uni par sprints de 2 semaines, chacun choisissait une fonctionnalité et prenait le temps nécessaire pour la réaliser avant de l'ajouter au jeu, que ce soit une heure ou un mois. Cette autonomie accrue a permis de réduire les conflits de versions liés à la modification simultanée de fichiers communs, et donc d'avancer plus vite sur chaque partie. Cependant cela rendait la vision globale de l'avancement plus difficile à appréhender.

2.3 Outils de gestion de projet

Pour s'organiser tout au long du projet, nous avons choisi d'utiliser l'application web Trello. Elle permet de créer un système de tableau Kanban, donc de gérer une liste de tâches avec un système de priorité et des colonnes "A faire", "En cours" et "Terminée". Nous utilisons aussi cette application pour écrire nos comptes rendus hebdomadaires. Ils permettent de tenir nos professeurs au courant de l'avancement actuel du développement du jeu mais aussi de réfléchir aux tâches que nous devrons prioriser la semaine suivante en fonction des retours de chacun et des difficultés rencontrées.

Pour les divers rapports écrits gardant une trace écrite de nos idées, nous avons utilisé la suite *Google* avec *Docs* et *Drive*. Le graphique des dialogues du jeu a été réalisé avec *Canva* car c'est une application facile à prendre en main et efficace pour des premiers jets peu complexes.

2.4 Répartition des rôles

Lors de la première phase, nous nous sommes répartis sur les quatre grandes parties identifiées lors de l'organisation :

— Modélisation : Enzo, Eric ;
— Scripting : Matéo, Eric-Nicolas, Clément, Maya ;
— Histoire : Eric ;
— Game design : Clément, Maya.

Au fil du projet, et particulièrement après les portes ouvertes, cette répartition a évolué : chacun étant devenu plus à l'aise, tout le monde a fini par toucher un peu à tout. Les rôles de la seconde phase se sont ainsi redéfinis autour de domaines plus précis :

— UI : Matéo (menus, écrans, éléments d'interface) ;

— Sound design : Eric (ambiances, bruitages, voix) ;
— UX : Eric et Matéo (parcours du joueur, tutoriels, confort et gestion de la motion sickness) ;
— Interactions et mains du personnage : Eric-Nicolas et Enzo (modélisation des mains et interactions entre les mains et les objets) ;
— Level design : Maya, Clément (conception de la seconde salle, création de nouveaux objets dessins) ;
— Modélisation et intégration : Clément, Eric-Nicolas, Eric (objets plus complexes connectés à un système d'événement).

3 Cycles de développement

3.1 Développement sur Blender

Pour ce qui est de la partie modélisation 3D, nous avons pu commencer assez tôt malgré les retards sur les TP en classe. Cela se justifie par le peu de temps qu'il nous a fallu pour nous mettre d'accord sur les designs ainsi que la direction artistique du jeu, mais aussi par le fait que les membres de notre groupe n'étaient pas étrangers à *Blender*, logiciel que nous avons utilisé pour modéliser nos assets. À l'aide de vidéos *Youtube* et nos connaissances nous avons progressé rapidement au cours des séances de projet en nous fixant toujours un objectif minimum de modélisation en début de séance.

Nous avons commencé par créer le personnage principal en parallèle des éléments de décor pour habiller la scène (les jouets, les petits objets, etc.), en gérant à chaque fois les textures pour que le rendu soit cohérent. Les principaux modèles ont été le berceau et le *Whack-a-Mole*. Il a fallu choisir intelligemment notre façon de modéliser ces objets de notre scène afin que le joueur comprenne intuitivement comment il pourrait interagir avec. En fin de compte, cela nous a forcé à penser nos modèles autant pour leur design que pour leur utilisation concrète en jeu. Tout au long de nos modélisations, nous devions garder en tête que nos objets doivent être géométriquement simples, c'est-à-dire composés de peu de triangles, pour que les casques de réalité virtuelle soient assez puissants pour faire fonctionner le jeu.

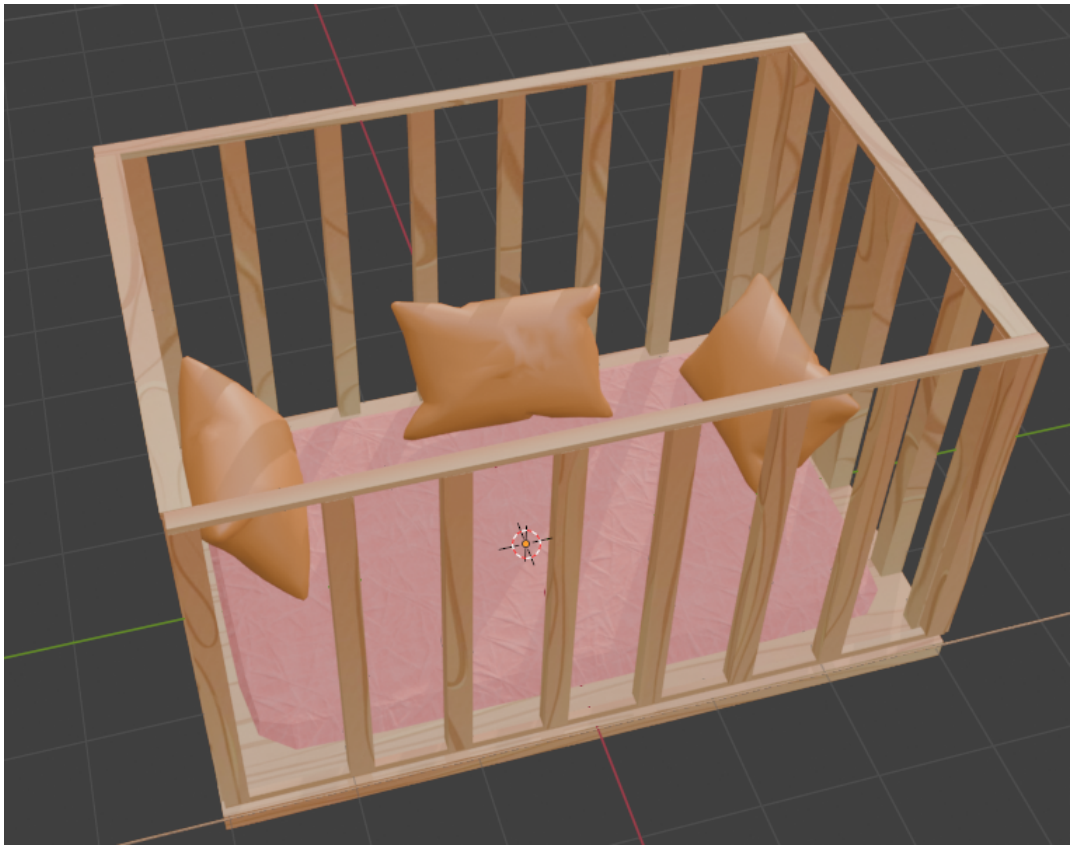


FIGURE 6 – Berceau modélisé sur *Blender*

Puisque l'avancement de la modélisation des objets de la première salle avait un peu d'avance sur la partie scripting, nous avons décidé de nous lancer sur un modèle bien plus complexe : le personnage principal. Il a été réalisé sur le cours de plusieurs séances en s'aidant de tutoriels sur *YouTube*. Nous n'avons pas encore pu l'implémenter dans la version actuelle du jeu mais nous savons qu'il sera très important pour le rendu final. En VR, si les mains du personnage principal ne sont pas modélisées et animées, *Unity* affiche simplement les manettes blanches du joueur. Cela nuit grandement à l'immersion. Pour le reste du modèle, nous allons essayer d'implémenter un miroir pour que le joueur puisse voir le personnage qu'il incarne.



FIGURE 7 – Personnage principal modélisé sur *Blender*

Les deux figures ci-dessus sont deux assets que nous avons modélisés. On peut y voir le berceau avec porte coulissante où apparaît le joueur au début du jeu ainsi que l'avatar du joueur représenté par un avatar humanoïde en *low-poly* (design avec volontairement peu de polygones). Nous avons modélisé nous-mêmes tous les objets présents dans notre jeu car les modèles en ligne ne correspondaient pas vraiment à notre vision pour le jeu. Voici la liste exhaustive des modèles que nous avons créés :

- Personnage principal ;
- Berceau ;
- Chaise en bois ;
- Table ;
- Balles de jonglage (inspiré d'un modèle déjà existant) ;
- Bibliothèque ;
- Charpente du toit ;
- Escalier en colimaçon ;
- Barrière pour l'étage ;
- Planétarium (avec animation) ;
- Veilleuse (avec animation) ;
- Marteau ;

— *Whack-a-Mole* (avec animation) ;
— Porte de séparation entre les salles (avec animation) ;
— Tuyau pour apporter les balles ;
— Pupitre ;
— Tapis pour enfant (à partir d'une image sur internet) ;
— Table basse ;
— Télévision ;
— Chaise en plastique.

Cette liste ne contient pas certains objets que nous n'avons pas utilisé, ni les murs et le sol réalisés directement sur *Unity*.

3.2 Développement Unity

3.2.1 Game Design et progression du joueur

Le game design de notre projet repose sur une progression linéaire structurée autour de plusieurs salles et mini-jeux, dont l'objectif est d'initier progressivement le joueur aux mécaniques fondamentales de la réalité virtuelle. Le joueur incarne un enfant évoluant dans un environnement contrôlé par une intelligence artificielle, ce qui permet d'intégrer les interactions VR de manière logique pour l'histoire.

La progression est construite sous forme de paliers successifs. Chaque salle introduit un nouveau type d'interaction : manipulation d'objets physiques, précision de lancer, puis logique de combinaison. Cette structuration permet d'éviter une surcharge cognitive pour un utilisateur novice en VR tout en maintenant une diversité de gameplay.

3.2.2 Mini-jeux et implémentation technique

Whack-a-Mole

Le *Whack-a-Mole* repose sur un système d'animation des taupes et de détection de collision avec un marteau manipulable en VR. Les animations des taupes sont activées de manière aléatoire via des scripts Unity, et leur état est géré par une logique simple (apparition, actif, disparition). Lorsqu'une collision est détectée entre le marteau et une taupe active, un événement est déclenché pour incrémenter le score et vérifier la condition de fin de jeu. Une fois l'objectif atteint, un événement global est envoyé au gestionnaire d'événements afin de signaler la complétion du mini-jeu.

Jeu de lancer de balles (Basket)

Ce mini-jeu repose sur le système de physique Unity, où le joueur doit lancer des objets dans un panier. Les balles sont générées depuis un tube et utilisent les composants physiques standards (`Rigidbody`, `Collider`) pour simuler une trajectoire réaliste.

La détection de réussite est effectuée via des `Trigger Colliders` placés dans le panier. Chaque entrée valide incrémente un compteur, et une fois le seuil atteint, un événement de complétion est déclenché via le système global.

Slot Machine

La machine à sous constitue le mini-jeu le plus complexe du projet en termes d'implémentation. Elle repose sur quatre roues indépendantes représentant chacune une suite de lettres. Le joueur peut interagir avec la machine via un levier, déclenchant la rotation des roues pour changer le mot affiché.

Le fonctionnement du système repose sur une banque de mots prédéfinis. Lorsqu'une partie est lancée, un mot est sélectionné aléatoirement à l'aide de `Random.Range`. Chaque lettre du mot est ensuite traitée individuellement : pour chaque roue, le programme recherche l'index correspondant à la lettre à afficher dans la liste des lettres disponibles sur cette roue. Cette étape permet de déterminer la position finale que devra atteindre chaque roue.

La rotation est réalisée à l'aide de coroutines *Unity* et de la fonction `Quaternion.Slerp`. Nous avons dû effectuer plusieurs rotations successives plutôt qu'une unique interpolation entre la position initiale et la position finale. En effet, une interpolation directe amènerait *Unity* à choisir le chemin de rotation le plus court entre les deux orientations. Dans certains cas, cela provoquait un désalignement visuel de la roue ou une rotation dans un sens inattendu. En découpant le mouvement en plusieurs tours complets avant l'arrêt final, nous conservons un comportement cohérent tout en garantissant que la roue s'aligne correctement sur la lettre cible ce qui permet au joueur d'observer progressivement le mot apparaître.

Une autre difficulté importante concernait l'insertion des cubes dans la machine. En réalité virtuelle, il est difficile pour le joueur d'aligner parfaitement un objet avec un emplacement précis, notamment à cause de l'absence de retour haptique réel et des imprécisions naturelles des mouvements de la main. Pour rendre cette interaction plus confortable, nous avons mis en place un système d'assistance à l'insertion : lorsque le joueur approche un cube d'un emplacement compatible, un hologramme apparaît à l'intérieur du logement afin d'indiquer visuellement la position attendue. Si le cube est suffisamment proche, le joueur peut simplement le relâcher : le système le repositionne automatiquement dans son emplacement final grâce à un mécanisme de *snap*. Cette approche évite au joueur d'avoir à réaliser un alignement parfait tout en conservant une sensation d'interaction naturelle.

Le système reste entièrement réversible. Un cube déjà placé peut être repris à tout moment par le joueur puis retiré de la machine afin de modifier la combinaison sélectionnée. La gestion de ces interactions a nécessité de nombreux tests afin de couvrir différents cas d'utilisation : cube approché sous différents angles, relâché trop tôt, présence de plusieurs cubes à proximité ou retrait d'un cube déjà validé. Dans ce dernier cas, le système

doit être capable de mettre à jour dynamiquement l'état de la combinaison en retirant la contribution du cube concerné. Par exemple, si un cube placé est retiré par le joueur, le compteur est décrémenté afin que la machine ne considère plus la combinaison comme valide.

Si les conditions de victoire sont remplies, un événement global est déclenché afin d'autoriser la progression du joueur.

3.2.3 Level Design : conception et refonte de la deuxième salle

Pour le design de la deuxième salle, plutôt que de prolonger simplement l'ambiance enfantine de la première, nous avons voulu créer une rupture volontaire : faire passer le joueur d'un environnement familier et coloré à quelque chose de plus oppressant, avant de le ramener vers un espace à nouveau plus léger. L'idée était d'installer une atmosphère mystérieuse et de donner au jeu une certaine profondeur narrative, sans pour autant l'expliquer explicitement au joueur.

C'est dans cette logique qu'est née l'idée du labyrinthe : un espace sans instructions, où le joueur doit trouver son chemin par lui-même. Pour renforcer l'aspect déstabilisant, nous nous sommes appuyés sur l'esthétique des *Backrooms* : éclairage froid et uniforme, couloirs répétitifs, absence de repères visuels forts. L'objectif n'était pas de faire peur au joueur, mais de le sortir de sa zone de confort et de créer un contraste marqué avec ce qu'il venait de traverser.

Pour marquer la transition vers la partie suivante, nous avons choisi d'utiliser un ascenseur. Une fois l'ascenseur arrivé à destination, le joueur débouche sur un étage lumineux et coloré, une aire de jeux pour enfants où l'attendent un *Simon Says* et un *Black Jack*. Ce retour à un environnement familier vient boucler ce moment de rupture, tout en gardant le joueur dans l'incertitude sur ce que le jeu cherche vraiment à lui faire vivre.

Sur le plan technique, la première version de la salle avait été montée directement dans *Unity*, ce qui posait plusieurs problèmes : murs mal alignés, géométrie irrégulière et impossibilité d'appliquer correctement les matériaux. Ce dernier point venait de la communication difficile entre *Blender* et *Unity* au niveau des textures, déjà rencontrée plus tôt dans le projet. Les couleurs et matériaux définis dans *Blender* reposent souvent sur des fonctions mathématiques que *Unity* ne sait pas interpréter directement, ce qui menait à des matériaux manquants ou saturés à l'import.

Nous avons donc entièrement reconstruit la salle sous *Blender* en blocs modulaires, avant de l'importer dans *Unity*. La solution au problème de textures a été de *bake* les matériaux avant l'export, c'est-à-dire de convertir les fonctions mathématiques qui le génèrent en images enregistrables que *Unity* peut ensuite appliquer simplement sur les modèles. L'éclairage a par ailleurs été un élément central de cette refonte : nous avons utilisé un système de *baked lighting* pour apporter une ambiance immersive, avec des lumières volontairement espacées afin de guider subtilement le joueur tout en renforçant

l'atmosphère du labyrinthe. En repartant sur une base modulaire propre et des textures bake, cette nouvelle version a réglé à la fois les problèmes de géométrie et de matériaux.

3.2.4 Architecture du projet et outils VR

Pour le développement du jeu, nous utilisons le moteur de jeu *Unity* 3D. Pour effectuer les tests dans les mêmes conditions qu'un joueur, nous avons utilisé le logiciel *Meta Quest Link* pour exécuter le jeu dans le casque. Sachant que les templates fournis par *Unity* couvrent seulement la réalisation de jeux 3D, il a fallu installer des packages pour gérer le backend XR — dans notre cas, les packages *OpenXR* et *OculusXR* — et les interactions du joueur (manettes, mouvement). Une scène de démonstration est fournie, ce qui nous a permis de commencer avec de bonnes bases.

3.2.5 Pipeline de création et intégration des assets

Après la modélisation d'un objet sur *Blender*, nous l'importons sur *Unity* puis il est placé dans la scène et transformé en préfabriqué (prefab). Les prefabs sont des objets réutilisables et modifiables que l'on place dans la hiérarchie du projet, cela nous permet d'éviter de modifier la scène, à la place on modifie le prefab. Une scène sur *Unity* est l'environnement de jeu dans lequel nos objets sont disposés. L'utilisation de prefabs nous permet d'éviter les conflits *Git* qui apparaissent lorsque plusieurs personnes modifient en même temps un même fichier car la scène est un fichier partagé par tous les membres du groupe. Dans la continuité de notre travail de modélisation, nous avons également animé certains éléments du décor afin de rendre l'environnement plus vivant et plus interactif. Nous avons notamment travaillé sur les portes et mécanismes mobiles en utilisant *Blender* pour créer les animations nécessaires, puis en exportant ces objets vers *Unity* sous forme de préfabs. Cette étape nous a permis d'assurer une intégration cohérente entre la phase de création des assets et leur utilisation concrète dans le gameplay.

Après l'intégration d'un asset, nous avons deux choix possibles : si c'est un objet de décor comme la figure 8, nous n'avons rien à faire dessus si ce n'est de le mettre au bon endroit, lui donner les bonnes proportions et des colliders approximatifs pour la physique.

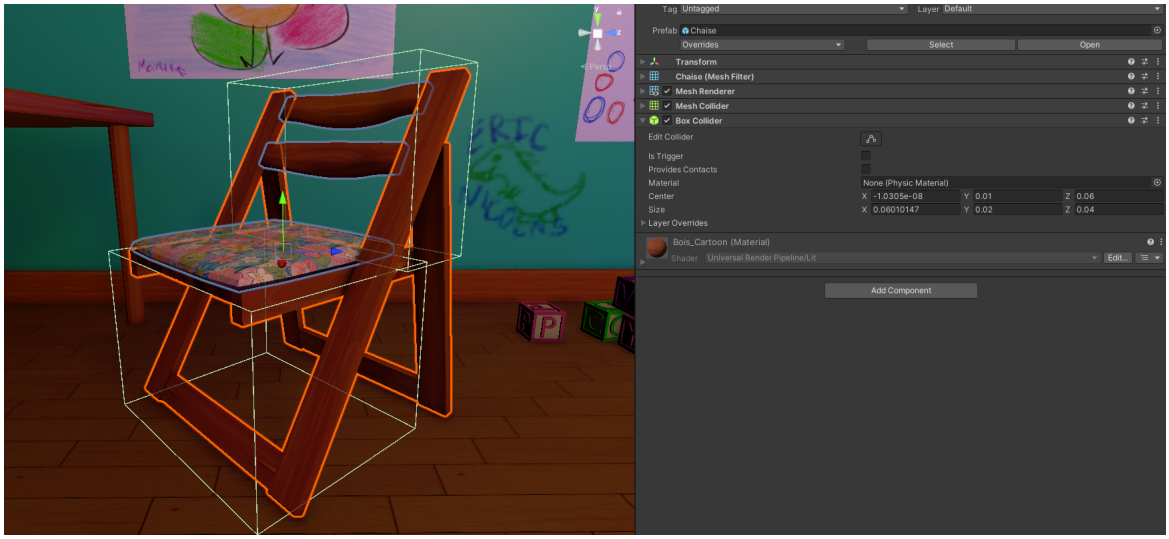


FIGURE 8 – Chaise de décoration dans la scène

En revanche, si c'est un objet qui sert au gameplay, davantage de travail est requis. Pour un objet attrapable, nous devons lui assigner les scripts fournis par les plugins XR (*XR Grab Interactable*, *Rigidbody*). Ces plugins permettent de simplifier la gestion des interactions ainsi que l'intégration de la VR.

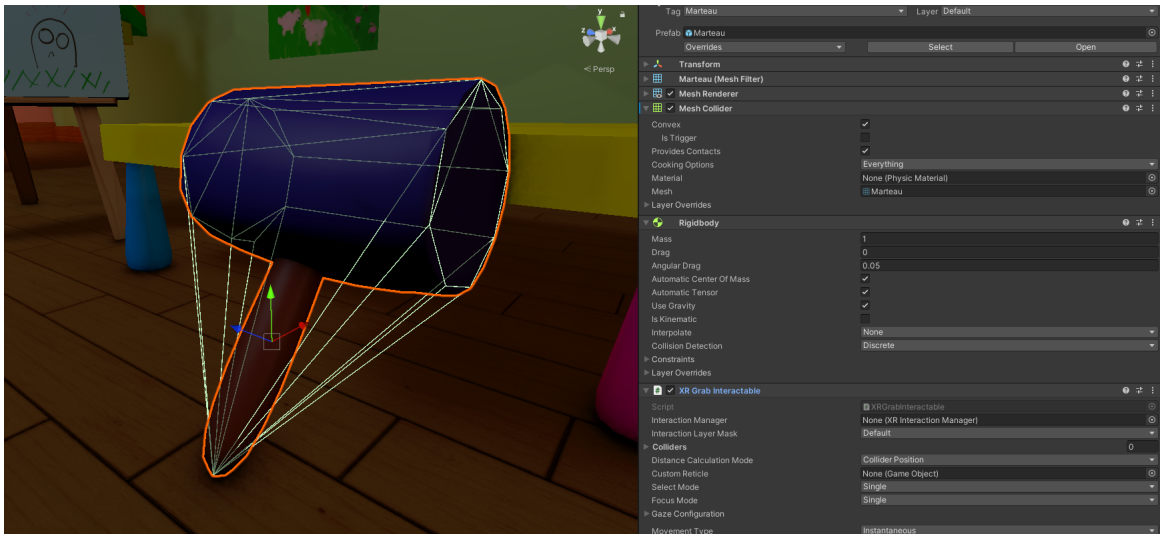


FIGURE 9 – Marteau pour le *Whack-a-mole*

Pour des objets plus complexes comme notre jeu *Whack-a-mole*, le système doit pouvoir reconnaître les interactions, par exemple un marteau qui touche une taupe et pas un autre objet. Il faut créer l'animation des taupes sur *Unity* pour pouvoir les utiliser dans un script (C#) et créer des événements pour signifier la fin du jeu et pouvoir l'afficher sur une télévision par exemple.

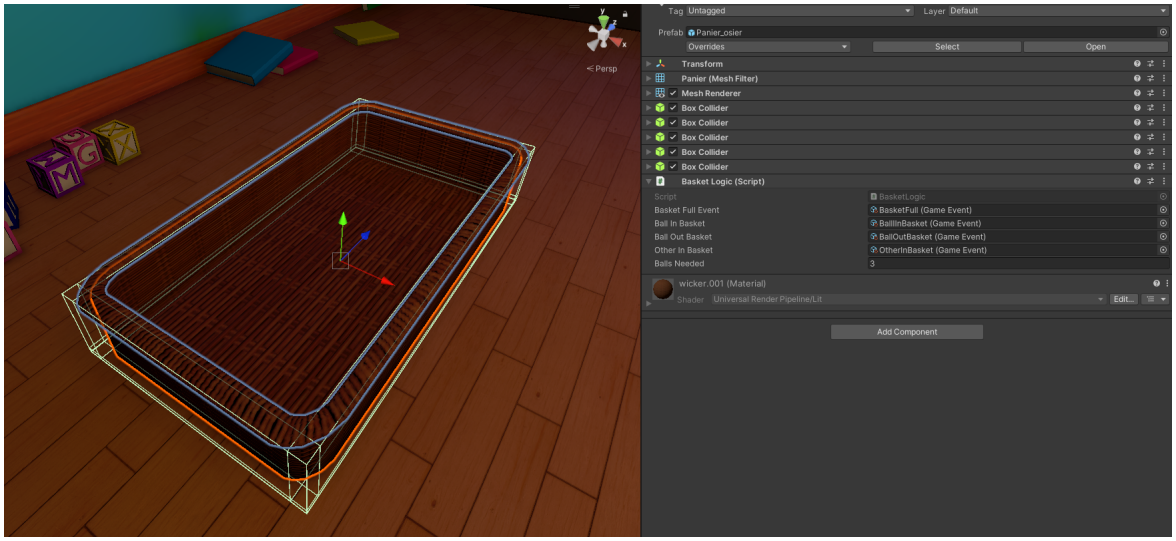


FIGURE 10 – Panier dans lequel les balles doivent être lancées

3.2.6 Système d'événements et architecture logicielle

Lorsque l'on code les interactions entre objets, plusieurs approches sont possibles. La plus directe consiste à écrire, sur un objet A, une fonction qui appellera une fonction sur un objet B au moment voulu. Le problème de cette méthode est qu'elle devient vite laborieuse à maintenir dès qu'il y a beaucoup d'interactions : chaque objet doit connaître les autres, ce qui crée de fortes dépendances. Nous avons donc opté pour une seconde approche, l'*observer pattern* (patron observateur).

Le principe repose sur des *game events*. Un *game event* est un canal de communication : un objet peut le déclencher pour signaler que quelque chose vient de se produire, sans jamais savoir qui va réagir. À l'inverse, n'importe quel objet peut se mettre à l'écoute de ce canal via un *listener* (écouteur), et exécuter une action dès que l'événement est émis. L'émetteur et les récepteurs sont ainsi totalement découplés : ils communiquent uniquement à travers l'événement, sans se connaître directement.

La figure 11 illustre ce fonctionnement sur un cas concret de notre jeu. Lorsque le joueur réussit à mettre une balle dans le panier, celui-ci déclenche l'événement « Balle dans panier ». Deux objets sont à l'écoute de cet événement : la télévision, qui affiche alors un message, et la porte, qui s'ouvre. Le panier n'a aucune connaissance de la télévision ni de la porte ; il se contente d'émettre l'événement, et chaque objet abonné réagit de son côté. Ajouter un nouveau comportement revient simplement à brancher un *listener* de plus sur l'événement, sans toucher au reste.

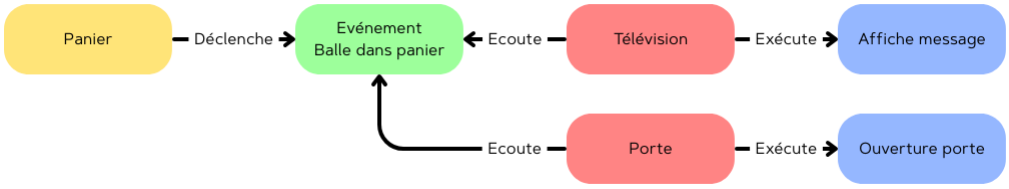


FIGURE 11 – Graphique d'explication de l'observer pattern

L'intérêt majeur est que tout le jeu est « branché » sur ce système d'événements, ce qui nous permet de développer en boîte noire. Nous pouvons par exemple créer l'événement « balle dans panier » et y brancher l'ouverture de la porte avant même que le panier ou la balle n'existent. Chacun peut donc travailler sur sa partie indépendamment, tant que les événements communs sont définis. C'est ce qui nous a permis, par la suite, d'ajouter les dialogues en quelques clics : il suffisait de les abonner aux événements déjà existants et fonctionnels pour qu'ils se déclenchent exactement au bon moment.

Pour centraliser tout cela, nous avons mis en place un objet maître qui gère l'ensemble des événements. Cela facilite l'accès à l'information pour les objets de la salle, et simplifie aussi notre travail de développeurs : si l'événement « balle dans panier » ne fonctionne pas comme prévu, nous savons immédiatement où chercher.

3.2.7 Menu et interfaces VR

Les menus ont été réalisés avec des objets directement présents dans le monde du jeu plutôt qu'avec une interface 2D classique affichée sur l'écran. Les boutons sont représentés par des objets 3D avec lesquels le joueur peut interagir grâce à un rayon émis depuis la manette permettant de viser les boutons.



FIGURE 12 – Menu actif dans la scène

Une séparation a été définie entre les interactions de menu et les interactions en gameplay. Les menus sont utilisables avec des rayons attachés aux manettes. Pendant le reste du jeu, toutes les interactions se font directement avec les mains virtuelles.

Cette séparation permet de distinguer les phases d'interfaces et les phases de gameplay, en gardant comme principe de rendre l'utilisation des menus assez simple. Lorsqu'un menu est ouvert, que ce soit au démarrage ou lorsque le joueur appuie sur la touche de pause, le joueur perd temporairement la possibilité d'interagir avec le reste du monde afin de centrer son attention sur l'interface. Le monde continue à tourner mais le joueur ne peut simplement plus interagir avec.

Afin de renforcer la lisibilité des menus et de donner un effet immersif supplémentaire, nous avons mis un système d'occultation de l'environnement (*fog of war*) autour du joueur lors de l'utilisation du menu. Cela crée une ambiance proche d'une mise en scène de pièce de théâtre où seul la zone importante reste éclairée. Cette approche est relativement simple à mettre en place techniquement, car elle ne demande qu'une simple forme 3D à avec une faible opacité, mais a prouvé être très efficace dans son effet visuel.

L'une de nos inspirations pour les menus en VR était *Ghosts of Tabor*. Ce jeu VR met également en place des menus dans l'environnement au lieu de simplement faire une interface 2D.

Pour ajouter un aspect amusant aux menus, tout en restant dans le thème, nous avons fait en sorte qu'à l'activation des boutons des menus, ils tombent par terre et se retrouvent ajoutés à l'environnement de jeu.



FIGURE 13 – Boutons du menu au sol

3.2.8 Sound Design

Le son remplit deux fonctions essentielles dans le jeu : l'immersion et le guidage du joueur. L'immersion passe par le feedback auditif. Le joueur doit sentir que ses interactions ont un effet concret sur le monde : lorsqu'une balle touche le sol, elle doit faire du bruit comme dans la réalité, et le marteau du *Whack-a-Mole* doit produire un impact sonore satisfaisant. Sans ce retour, les actions paraissent vides et l'immersion se brise.

Le son sert aussi à diriger le joueur de manière naturelle. Plutôt que de surcharger l'interface d'indications visuelles, nous nous appuyons sur les dialogues : une voix qui demande l'attention du joueur et l'implique dans l'action, tout en le guidant sans alourdir l'écran. Ces dialogues sont branchés sur le système d'événements décrit précédemment, ce qui leur permet de se déclencher exactement au bon moment.

Côté implémentation, nous avons centralisé toute la gestion audio dans un objet unique, l'`AudioSystem`, sur le même principe que le système d'événements. Il regroupe l'ensemble des sons au même endroit et gère leur déclenchement ainsi que leurs paramètres (clip, volume, pitch). Cette centralisation simplifie le développement : tous les sons sont accessibles depuis un seul point, et un éventuel problème sur l'un d'eux se localise immédiatement.

Nous distinguons deux types de sons. Les sons 2D, comme les dialogues et les bruits de fond, sont joués de manière uniforme sans position dans l'espace. Les sons 3D, comme le rebond d'une balle ou le « bonk » du marteau, utilisent les bibliothèques intégrées de *Unity* pour être spatialisés : leur volume et leur provenance dépendent de la position du joueur, ce qui renforce la sensation de présence. L'ensemble des bruitages provient de bibliothèques de sons libres de droits, tandis que les dialogues ont été écrits à la main puis interprétés par Maya.

4 Contraintes et immersion

La réalité virtuelle possède des contraintes très différentes des médias standards. Habituellement, l'utilisateur reste physiquement immobile devant un moniteur, tandis qu'en VR il est directement plongé dans l'environnement et perçoit les mouvements comme des mouvements naturels.

Une notion importante est celle de la *motion sickness*, qui définit les effets négatifs que l'utilisation de la Réalité Virtuelle peut avoir sur un utilisateur. Ces effets négatifs peuvent être :

— Vertige ;
— Vomissement ;
— Mal de tête.

Une mauvaise gestion des déplacements, de la caméra ou des interactions peut avoir des effets négatifs sur l'utilisateur, jusqu'à atteindre des pertes de repères ou du *motion sickness*. L'un de nos principaux critères de réussite dans ce projet a été de concevoir une expérience immersive tout en restant confortable pour les utilisateurs néophytes à la Réalité Virtuelle.

4.1 Déplacements et confort utilisateur

Le système de déplacement que nous avons choisi est le déplacement fluide avec une rotation de la caméra par angle (*snap*). L'utilisateur peut tout de même bouger la tête naturellement mais il a le choix d'utiliser un stick analogue afin de se déplacer dans l'environnement ou d'effectuer une rotation de la caméra. Nous avons fait de nombreux tests et ce que nous avons identifié est que la rotation de la caméra "fluide" augmente fortement les risques de motion sickness. La rotation par angle n'est pas parfaite mais elle réduit ce risque.

La vitesse de déplacement et de rotation ont également été ajustées après de nombreux tests. Une vitesse trop élevée augmentait les risques de motion sickness tandis qu'une vitesse trop faible rendait l'expérience ennuyeuse.

Nous avons également évité certaines mécaniques pouvant causer de l'inconfort :

— Les déplacements forcés ;
— Les chutes ;
— Les accélérations brutales ;
— Les mouvements de caméra non voulus par l'utilisateur ;
— Les situations pouvant donner une sensation de vertige.

Avec ces choix, le joueur garde constamment le contrôle de ses mouvements, ce qui contribue au confort de l'expérience.

4.2 Retours visuels, sonores et haptiques

En Réalité Virtuelle, les retours (feedbacks) sont essentiels pour permettre à l'utilisateur de comprendre rapidement si une interaction a eu un effet sur le système. Nous avons donc dédié une certaine attention aux retours visuels, sonores et haptiques.

Concernant les retours visuels, lors de la navigation dans les menus, le rayon de la manette s'étend lorsqu'il rentre en contact avec un bouton afin de signaler le bouton ciblé. Également, quand on passe au dessus des différents objets interactifs, ils s'illuminent. Cela permet d'avoir une meilleure compréhension des interactions.

Les manettes ont aussi des retours haptiques (vibrations) sur certaines interactions afin d'indiquer à l'utilisateur l'interaction avec les objets virtuels.

Enfin, le son joue un rôle clé de confirmation. Au-delà de l'ambiance, chaque action importante s'accompagne d'un retour sonore qui indique au joueur que son geste a bien été pris en compte : une balle qui rebondit avec un bruit réaliste, ou un petit « boop » lorsqu'il attrape un objet, par exemple. Sans ce retour, le joueur peut douter d'avoir réellement saisi un objet ; le son lève cette ambiguïté de façon immédiate et naturelle. Nous avons d'ailleurs jugé que ces retours sonores étaient moins agressifs que des retours lumineux systématiques, ce qui nous a permis de préserver une atmosphère cohérente.

4.3 Interactions physiques et immersion

L'un des aspects centraux de notre projet est l'utilisation de la physique pour renforcer l'immersion. La majorité des objets présents dans le jeu peuvent être attrapés et manipulés directement par le joueur. Cette liberté dans les interactions permet de renforcement l'immersion dans le monde virtuel.

Nous avons aussi configuré la majorité des objets interactifs afin qu'ils puissent être attrapés sous différents angles, sans avoir de position prédéfinie. Ce choix permet d'éviter l'effet très artificiel que peut parfois avoir la Réalité Virtuelle. Nous avons cependant laissé des positions prédéfinies pour certains objets tels que le marteau.

Cette liberté d'interaction est notamment inspirée de jeux comme *Boneworks* qui se veulent orientés sur la manipulation réaliste des objets et les réactions physiques du monde. Nous n'avons cependant pas simulé un effet de poids que peut présenter ce jeu de par la difficulté d'implémentation ainsi que le coût de calcul sur notre matériel.

5 Retours utilisateurs

Tout au long du projet, nous avons fait tester le jeu à plusieurs personnes, notamment lors des journées portes ouvertes de l'ENSIM. Les profils étaient variés — lycéens, étudiants, parents — et pour la grande majorité, il s'agissait de personnes n'ayant jamais essayé la réalité virtuelle, quelques-unes ayant une expérience légère du médium. Ces sessions de test nous ont permis de recueillir des retours concrets sur chaque aspect du jeu, en particulier sur l'accessibilité et la prise en main.

5.1 Déplacements et confort

De manière générale, la vitesse de déplacement a été jugée satisfaisante. En revanche, la rotation par angle a été perçue comme trop brutale par plusieurs testeurs. C'est un parti pris volontaire dont nous connaissons les défauts, mais nous avons jugé que notre choix restait le bon malgré les doutes des utilisateurs. La rotation par angles peut paraître bizarre pour la plupart des utilisateurs mais elle est meilleure car elle provoque moins de motion sickness.

5.2 Guidage et prise en main

Le retour le plus fréquent portait sur le manque d'indications claires, en particulier en début de jeu. Des actions qui nous semblaient évidentes, comme se déplacer avec le joystick ou simplement tourner la tête, n'allaient pas du tout de soi : l'ouverture de la porte initiale et l'utilisation des joysticks n'étaient pas intuitives pour les nouveaux joueurs. Il fallait souvent expliquer en même temps les bases de la VR et les mécaniques du jeu, ce qui représente une charge importante pour quelqu'un qui découvre le médium. Les testeurs des portes ouvertes ont par exemple trouvé que les dessins explicatifs présents dans la salle n'étaient pas assez visibles pour être utiles sans aide extérieure.

Tout cela nous a confirmé la nécessité d'un vrai tutoriel en début de partie et d'un guidage visuel repensé. Les dessins ont été redessinés pour être plus clairs et repositionnés à des endroits plus naturels dans le champ de vision du joueur, afin qu'il comprenne ce qu'on attend de lui sans assistance. Nous avons aussi ajouté davantage de tutoriels, notamment des animations de manettes montrant les boutons enfoncés pour expliquer clairement comment interagir. Nous avons enfin envisagé l'ajout de *spotlights* sur certains éléments clés pour attirer l'attention du joueur sans alourdir l'interface.

5.3 Mini-jeu du panier

Le lancer de balles s'est révélé trop difficile pour plusieurs testeurs, la mécanique de lancer en VR étant peu naturelle pour des novices. Différentes pistes ont été envisagées, comme faire rouler les balles plutôt que les lancer, ou encore limiter le nombre de balles disponibles à trois en début de partie tout en générant de nouvelles balles en cas de perte. Ces ajustements visaient à rendre le mini-jeu plus accessible sans le vider de son intérêt.

5.4 Interactions et objets

Plusieurs problèmes liés aux interactions physiques ont été identifiés. Les objets attrapables pouvaient parfois traverser d'autres objets (*clipping*) lors d'un grab, ce qui nuisait à l'immersion. Un problème de joueur bloqué lors d'un grab lié au joystick avait également été remonté, mais était déjà corrigé au moment des tests. Un retour haptique sur le *Whack-a-Mole* a aussi été suggéré pour mieux signaler les impacts. Des problèmes de mise à l'échelle persistaient également sur ce mini-jeu, ainsi que sur certains éléments d'interface comme un écran jugé trop haut.

5.5 Ambiance et level design

Les couleurs et l'ambiance générale du jeu ont été bien reçues. Les testeurs ont apprécié la présence de petits objets décoratifs éparpillés dans la salle, qui encouragent l'exploration et augmentant le temps de jeu. Quelques ajustements ont été suggérés, comme baisser le plafond pour renforcer la sensation d'échelle adaptée à un enfant.

6 Difficultés rencontrées

La création et le début du projet n'a pas été sans embûche, notamment car les premières séances ont eu lieu avant les premiers travaux pratiques de VR.

6.1 Exports

Lors de nos premiers exports, nous avons été confrontés à des problèmes d'absence de matériaux.

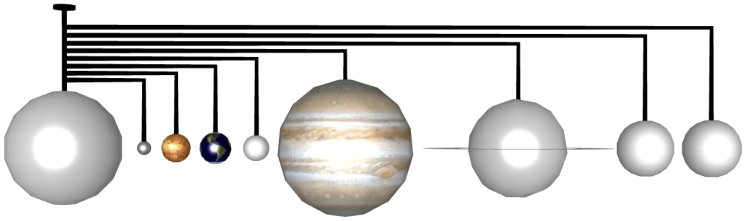


FIGURE 14 – Visualisation d'un modèle en `fbx`, format majoritaire dans le jeu

Les exports au format `gltf` ont bien les matériaux assignés mais ils sont saturés ou transparents.

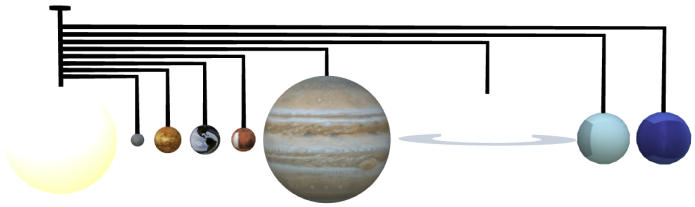


FIGURE 15 – Visualisation d'un modèle en `gltf`

Pour régler ces problèmes, nous avons eu l'aide de M. Coupry qui nous a appris que nous devions "bake" nos textures avant de les exporter de *Blender* vers *Unity*. Cela signifie par exemple (tous les cas sont différents) que plutôt que d'avoir une fonction mathématique pour définir une couleur sur un objet, nous devions la transformer en image enregistrable dans un dossier. *Unity* pouvait donc ainsi prendre cette image et l'appliquer sur notre objet.

6.2 Connexion avec le casque

Pour assurer la liaison entre le casque et le PC, il faut passer par Meta Quest Link, cependant pour se connecter, le logiciel demande une autorisation par Internet, ce que le proxy de l'université refuse. Notre solution a été de connecter les ordinateurs à des points d'accès ou des modems.

6.3 Erreurs de compilation

Parfois en ouvrant le projet nous étions confrontés à des erreurs de compilation. La solution que nous avons trouvée est de supprimer le dossier *Library*. Cela nécessitait donc une recompilation complète des bibliothèques par *Unity* qui nous faisait perdre une dizaine de minutes à chaque fois que cela arrivait.

6.4 Conflits

Sur *Unity*, une scène est référencée par un fichier unique. Si plusieurs personnes modifient ce fichier de leur côté, les références internes varieront et mèneront à un conflit. C'est pourquoi nous avons dû séparer les types de modifications : ajout de modèles, programmation, etc.

7 Étude d'impact environnemental

Comme montré dans l'article [1], le langage C# consomme plus d'énergie que des langages compilés comme C ou C++. Cela s'explique entre autres par la présence d'un ramasse-miettes, qui s'occupe du recyclage de la mémoire utilisée par le programme, et l'utilisation d'une machine virtuelle (l'environnement qui exécute le code). Malheureusement nous ne pouvions pas changer de langage car il est étroitement intégré à *Unity*, les langages tiers ne permettant de communiquer qu'avec une partie de l'API proposée. Les seules autres possibilités de réduire l'impact du langage sont d'optimiser des algorithmes et éviter que des instructions coûteuses (comme les logs, les calculs physiques) ne s'exécutent à chaque frame.

Un moteur de jeu comme *Unity* qui, bien que présentant beaucoup de fonctionnalités qui permettent de créer des jeux plus facilement, aura toujours un surcoût par rapport à un jeu réalisé de zéro, sans aucun moteur de jeu. Pour des raisons de temps et de familiarisation avec l'utilisation d'outils comme le *XR Interaction Toolkit*, cette option n'était pas concevable. D'un autre côté, *Unity* utilise une compilation incrémentale, recompilant uniquement les fichiers modifiés. C'est un gain de temps et d'énergie mais qui nécessite plus de mémoire de stockage, compromis que l'on retrouve également pour l'optimisation d'algorithmes.

Concernant l'utilisateur, l'*observer pattern* permet de minimiser le nombre d'appels CPU. La taille des ressources était également un autre enjeu : des textures avec un plus grands nombres de triangles serait certes plus lisses et plus détaillés, mais surtout plus lourds et ralentiraient le jeu. Il en va de même pour des *shaders* (programmes paramétrant une partie du rendu) complexes.

8 Conclusion

Nous sommes globalement satisfaits du résultat obtenu au terme du projet et pensons que nous avons répondu aux objectifs que nous nous étions fixés au début du développement. Le jeu propose aujourd'hui une expérience VR cohérente permettant à un utilisateur novice de découvrir progressivement les principales mécaniques de la réalité virtuelle : déplacements, interactions physiques, manipulation d'objets et résolution de mini-jeux. L'univers visuel du projet a également réussi à se construire autour d'une identité artistique mêlant esthétique enfantine et ambiance plus mystérieuse, notamment grâce au travail réalisé sur les modèles 3D, l'éclairage et le level design.

Au fil du projet, nous avons également pu approfondir plusieurs aspects techniques liés au développement VR, comme l'intégration d'assets *Blender* dans *Unity*, la gestion des interactions avec le *XR Interaction Toolkit*, la conception d'un système d'événements découplé ou encore les problématiques de confort utilisateur et de *motion sickness*. Les nombreux retours obtenus lors des journées portes ouvertes nous ont permis d'identifier les points forts de notre jeu mais aussi les éléments nécessitant des améliorations, en particulier concernant le guidage du joueur et certaines interactions physiques.

Ce projet nous a permis de découvrir concrètement les contraintes du développement en réalité virtuelle, aussi bien sur les plans techniques qu'artistiques et ergonomiques. Il nous a également appris à travailler en équipe sur un projet mêlant programmation, modélisation 3D, game design et expérience utilisateur. Au-delà du résultat final, cette expérience constitue pour nous une première approche concrète du développement de jeux VR et des problématiques liées à l'immersion interactive.

Références

[1] Rui Pereira et al. “Energy efficiency across programming languages : how do energy, time, and memory relate ?” In : *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*. SLE 2017. Vancouver, BC, Canada : Association for Computing Machinery, 2017, p. 256-267. ISBN : 9781450355254. DOI : 10.1145/3136014.3136031. URL : https://doi.org/10.1145/3136014.3136031.